

Formal Verification of Onion Franking Using Tamarin

Kayla Casey
University of North
Carolina at Chapel Hill
kaylaren@cs.unc.edu

Sarah Haddix
University of North
Carolina at Chapel Hill
sarabh@unc.edu

Bao-Nhi Vu
University of North
Carolina at Chapel Hill
baonhi@cs.unc.edu

Abstract—This paper presents a formal verification of key security properties in the “Onion Franking” abuse reporting scheme for metadata-hiding messaging systems. Using the Tamarin Prover, a tool for symbolic modeling and analysis of security protocols, we formally model a simplified version of the Onion Franking protocol and verify its correctness, unforgeability, and accountability guarantees under realistic adversary models. To build confidence in our methodology, we first model and verify the One-Time Pad (OTP), a classical encryption scheme with proven security properties that serves as a toy example. We then translate of Onion Franking into Tamarin and define lemmas corresponding to the security properties. Our work offers a first step toward providing machine-checked assurance for protocols designed to balance privacy and abuse mitigation in end-to-end encrypted messaging systems.

I. INTRODUCTION

The increasing adoption of end-to-end encrypted (E2EE) messaging systems has raised new questions about how to balance user privacy while providing moderation and abuse prevention. Onion Franking, a cryptographic protocol proposed by Gregoire, Pierce, and Eskandarian (2024) [1], enables verifiable abuse reports in E2EE metadata-hiding communication systems. This setting entails that clients are able to send messages to each other without the messaging platform seeing the contents of the messages or any associated meta-data (timestamps, identifying information of senders and receivers). The Onion Franking protocol allows clients to report abusive messages without disclosing any message data about non-reported messages [1]. The balance between user privacy and abuse prevention is difficult to achieve and has serious consequences: if anonymity is broken, users lose trust in the system; if abuse reporting fails, harmful actors exploit the platform with impunity.

Mindful of these consequences, we turn to formal verification to provide additional assurance of critical security properties. Formal verification allows us to prove that a model of a cryptographic protocol satisfies certain security properties under a precise threat model. We use the Tamarin Prover, a symbolic model checking and analysis tool, to carry out this verification. Tamarin Prover models the protocol as a set of rewriting rules and allows for the specification and automated proof of security properties expressed in first-order logic [3].

Our goal is to model the V.1 Construction of Onion Franking in Tamarin and verify its key properties: correctness, unforge-

ability, and accountability. These properties ensure that valid reports are always verifiable, that abuse reports cannot be forged, and that malicious actors cannot escape responsibility [1]. We begin by modeling a simpler protocol that is proven to be secure if used correctly, the One-Time Pad (OTP) [5], to become fluent in Tamarin’s syntax and logic. This step helps validate our approach and provides a clear baseline. We then proceed to encode Onion Franking’s V.1 Construction and formulate lemmas expressing the desired security guarantees.

This paper documents our progress toward modeling Onion Franking, presents our work on the OTP cipher as a toy example, and outlines the verification strategy we will apply. If successful, our work will provide formal assurance that Onion Franking resists key attacks within its intended threat model and offers a robust framework for abuse accountability in anonymous messaging systems. Additionally, our work provides both cryptographic researchers and system designers with a concrete example of how to model and verify the security properties of the protocol they are working on, which would both strengthen the veracity of research and the trust that users have in the systems they interact with.

II. BACKGROUND

A. The Tamarin Prover

The Tamarin Prover is a formal verification tool designed for modeling and analyzing the security properties of security protocols. It allows users to define protocol steps, specify an adversary model, and express desired security properties. Tamarin provides both automated and interactive modes for verification. If the tool terminates and the properties are satisfied, it returns a proof; if the properties are violated, it provides a counterexample representing a possible attack trace. Protocols are modeled using a multiset rewriting rule language, allowing fine-grained modeling. Tamarin includes a built-in Dolev-Yao adversary model [5], under which the adversary has full control of the network—intercepting, modifying, and replaying messages. Our project uses Tamarin to symbolically model and verify the Onion Franking protocol against its intended security guarantees.

B. One-Time Pad Cipher

The One-Time Pad (OTP) [4] is a classical encryption scheme that provides perfect secrecy, meaning an attacker

learns nothing about the plaintext from the ciphertext alone. This cipher relies on the assumption that the key used is truly random, as long as the message is, and is used only once. The OTP works by XOR-ing each bit of a plaintext message with a corresponding bit from the key. Decryption requires the recipient to know the exact key and simply apply XOR again to the ciphertext to retrieve the original plaintext. While the OTP is impractical for large-scale use due to its strict requirements, its simplicity and ideal security properties make it an effective toy model for learning and testing formal verification tools like Tamarin.

C. End-to-End Encryption (E2EE)

End-to-end encryption is a method of secure communication that hides messages from the platform handling communication, meaning only a sender and receiver can read their messages [1]. In E2EE systems, messages are encrypted on the sender's device and decrypted on the recipient's device, with no intermediate server or third party able to access the plaintext. This protects message contents even if the infrastructure is compromised. E2EE is a foundational principle in private messaging systems and is a critical assumption in protocols like Onion Franking, which aim to add accountability and strengthen privacy guarantees.

D. The Onion Franking Protocol

Onion Franking is a cryptographic protocol designed to support abuse reporting in privacy-preserving messaging systems that use onion routing. Onion routing is a mechanism of layering encryptions to hide sender and receiver identities as a message passes through a system of servers. Onion Franking introduces a mechanism for a sender to include a verifiable, hidden message authentication tag (MAC tag) within the message. If a recipient wishes to report abuse, they can extract this tag and present it to a moderator, who can verify its authenticity without learning the sender's identity. The system ensures that only the moderator can perform this verification, preserving deniability, and that no false reports can be constructed, preserving unforgeability. Onion Franking builds on cryptographic primitives such as commitments, signatures, and public key encryption, combining them in a novel way to enforce accountability without compromising user privacy.

E. Cryptographic Primitives in Onion Franking

Modern cryptographic protocols are built from well-defined primitives that achieve security properties through mathematical hardness assumptions. Onion Franking employs cryptographic primitives, including:

- *Public-Key Encryption (PKE)*: Allows users to encrypt messages using a recipient's public key to form a ciphertext that can only be decrypted by the corresponding secret key of the receiver.
- *Digital Signatures*: Provide authenticity of users and integrity of information. A sender will sign a message with their private key, which can be verified by anyone

holding the sender's public key. This ensures that a message has not been altered and that recipients can verify senders.

- *Commitment Schemes*: Enable a user to commit to a value while keeping it hidden, with the ability to reveal it later. A commitment can be both hiding, meaning that it reveals no information, and binding, meaning that it cannot be changed after committing.
- *Zero-Knowledge Proofs (ZPKs)*: Allow a prover to convince a verifier that a statement is true without revealing any information about the statement itself.
- *Hash Functions (modeled as Random Oracles)*: Provide deterministic mappings from arbitrary-length inputs to fixed-size outputs. In the random oracle model, the hash function is treated as a black box that outputs random values for new inputs.

These primitives are often used as the building blocks for other complex cryptographic protocols, and are verified for the security properties they provide. Tamarin allows us to reason symbolically about how these primitives interact, which we will use to compose the V.1 Onion Franking protocol.

F. Security Notions in Onion Franking

We consider the following security notions, as defined in the Onion Franking protocol [1] to formally verify:

- *Correctness*: If an honest sender generates a franking tag and an honest recipient reports it correctly, then the moderator should accept the report.
- *Unforgeability*: A malicious client should not be able to forge a report for a message it did not send, even when colluding with malicious servers. This ensures that no user can falsely blame another for a message they did not send.
- *Accountability*: If a message is sent through the system, it should be possible to verify that it was sent, even if the sender deviates from the protocol. This prevents malicious clients from escaping responsibility.
- *Deniability*: No third party, including compromised servers, should be able to verify a message or report; only the moderator can do so. This maintains the privacy and deniability of communication outside the abuse reporting context.
- *Confidentiality*: The integration of the Onion Franking mechanism must not compromise the confidentiality or metadata-hiding guarantees of the underlying messaging protocol, except in the case of a reported message.

These properties are vital to Onion Franking's goal of abuse reporting while preserving the core privacy protections of end-to-end encrypted communication. It is important to note that the exact definition of security properties will vary across different cryptographic protocols.

III. PROBLEM STATEMENT

We aim to verify the security properties of correctness, unforgeability, and accountability for the V.1 Construction of Onion Franking using Tamarin Prover.

Our verification is based on the threat model defined in the Onion Franking paper [1]. The protocol is designed to defend against adversaries who may act as malicious clients, sending abusive messages and attempting to generate false reports, and intercept, modify, delay, and replay network communication (i.e., full control of the network, as assumed by the Dolev-Yao model [6]). Tamarin’s built-in adversary model closely matches this setting. It assumes an attacker with complete control over the communication channel but unable to break cryptographic primitives (e.g., cannot guess private keys or invert hash functions without explicit modeling).

We do not consider global passive adversaries (e.g., full surveillance), side-channel attacks, or cryptographic breaks. These are out of scope for the Onion Franking paper [1] and Tamarin model. The verification goal is to ensure the cryptographic protocol functions as expected against realistic threats, making private messaging secure and accountable. Verification of the security protocols of deniability and confidentiality will assume an adversary that colludes with servers within the messaging system. However, these security protocols are a stretch goal, see ‘Plan of Work’.

IV. APPROACH

Our approach involves using Tamarin Prover to verify Onion Franking’s key security property satisfiability. To build familiarity with Tamarin, we began by modeling and verifying the One-Time Pad (OTP) encryption scheme as a toy example. OTP allows us to explore how cryptographic operations such as XOR and key freshness are represented in Tamarin’s multiset rewriting syntax, as well as how security properties are encoded as lemmas in first-order logic. We modeled OTP using three rules: key generation, encryption, and decryption. The key was defined as a fresh value, and the encryption rule combined the plaintext with the key using XOR. The lemma stated that an adversary cannot learn the plaintext if the key is unknown, capturing the secrecy property of OTP. Successfully running this model helped us understand how to use Tamarin’s GUI, debug syntax errors, and interpret proof results.

Based on this foundation, we are in the process of modeling Onion Franking by creating a symbolic representation of the protocol’s key roles (sender, recipient, moderator), message formats (including commitments and signatures), and verification logic. We will add support for correctness, unforgeability, and accountability, and have reach-goal properties of deniability and confidentiality. Each will be expressed as a Tamarin lemma and tested under the defined threat model.

An anticipated major challenge is modeling complex properties, like unforgeability and deniability, which require reasoning about what different entities can or cannot infer. Tamarin is less suited to these equivalence-based notions, though it does support observational equivalence through diff-equivalence lemmas. If time permits, we will attempt to model deniability using this feature, though this will likely require significant abstraction and simplification.

Additionally, Onion Franking relies on cryptographic primitives such as commitments, hash functions, and digital signa-

tures. We plan to represent these using symbolic abstractions, such as uninterpreted functions and equations, consistent with the random oracle and Dolev-Yao models [5].

Diagrams illustrating our current OTP model and message flow of Onion Franking will be included in the final paper to aid understanding. Visual aids will clarify how messages are constructed, passed through the system, and verified, and how Tamarin tracks the state and knowledge of different participants. Our work will provide a reusable formal model of Onion Franking and help identify potential pitfalls in designing accountable anonymous communication systems.

V. EVALUATION

Our evaluation focuses on formally verifying the One-Time Pad (OTP) as a toy protocol and then applying the same methodology to the V.1 Construction of Onion Franking. For OTP, we have confirmed that Tamarin can prove the secrecy property when the key remains unknown. Success is defined as Tamarin terminating with a proof or counterexample, depending on whether the security property holds. For Onion Franking, we will evaluate whether Tamarin can verify correctness, unforgeability, and accountability under our defined threat model. Success will be measured by whether Tamarin can exhaustively analyze the protocol without error and whether the results align with the security guarantees claimed in the original paper. We will document any discovered attacks or limitations in modeling and interpret their impact on the protocol’s security claims.

VI. PLAN OF WORK

From the project proposal to project update, our minimum goal is the same, our expected goal has been clarified regarding the protocol we wish to verify and security properties, and our reach goal has been updated to be more realistic.

Since the project proposal, we have met our minimum goal of becoming familiar with the Tamarin Prover, understanding the Onion Franking protocol, and successfully modeling and verifying the One-Time Pad (OTP) cipher to confirm our grasp of the tool and symbolic verification. This work has prepared us to begin modeling the V.1 Construction of Onion Franking in Tamarin. Our updated expected goal is to model this construction and formally verify its core security properties: correctness, unforgeability, and accountability. Our reach goal is to extend this model to cover additional properties such as deniability and confidentiality, and, if time permits, verify the optimized V.1 Construction. An additional stretch goal is to explore the more complex variants described in the original paper, such as those involving trap messages or zero-knowledge proofs.

These changes were enabled by the work we’ve done so far with Tamarin and the Onion Franking protocol, which has given us a better understanding of both the work we wish to verify and the tools with which we aim to do so.

VII. STATE OF THE ART

Formal verification tools, like Tamarin, are increasingly used to uncover flaws in protocols that may not be evident through unassisted reasoning or empirical testing. As cryptographic systems grow in complexity and are deployed at scale, formal verification offers a principled method for ensuring their correctness and resilience.

A. Symbolic Verification of Cryptographic Protocols

Symbolic modeling verification tools, such as Tamarin Prover, are widely used to verify security properties of cryptographic protocols [2]. Basin et al. [2] describe Tamarin’s core strengths, including its constraint solving engine, rule-based multiset rewriting semantics, and ability to verify trace properties such as secrecy, authenticity, and resistance to replay attacks. Our work builds on this foundation by applying Tamarin to a new protocol domain: anonymous, abuse-reportable messaging systems.

B. Verification of Complex and Real-World Protocols

In follow-up work, Basin et al. [3] used Tamarin to verify real-world protocols such as 5G-AKA and EMV card payment systems. These studies illustrate Tamarin’s scalability and adaptability to complex, stateful designs involving diverse adversarial environments. Our research applies these verification techniques to Onion Franking and identifies how formal guarantees about accountability and abuse reporting can be proven within symbolic models.

C. Security in Private Messaging Systems

Few symbolic verification efforts target protocols designed for metadata-hiding and user accountability in anonymous messaging systems. Onion Franking introduces new security goals, such as verifiable abuse reports and sender accountability under anonymity constraints. Our work contributes to this sub-area by being, to our knowledge, the first to attempt of formal symbolic verification to core security properties of abuse reporting mechanisms in metadata hiding systems using Tamarin. This represents an application of established tools to a underexplored protocol design space.

VIII. WHO DID WHAT

In developing the proposal, Kayla played a key role in crafting the introduction, background, threat model, approach, evaluation, plan of work, and references sections. Bao-Nhi contributed significantly to the introduction, state of the art, threat model, evaluation, plan of work, “who did what” section, and references. Sarah contributed to the problem statement, introduction, and plan of work, and provided essential support through overall editing and proofreading, ensuring that the proposal was cohesive and aligned with the required guidelines.

REFERENCES

- [1] M. Gregoire, M. Pierce, and S. Eskandarian, “Onion franking: abuse reports for mix-based private messaging,” Cryptology ePrint Archive, Paper 2024/1965, 2024. [Online]. Available: <https://eprint.iacr.org/2024/1965.pdf>
- [2] D. Basin, C. Cremers, J. Dreier, and R. Sasse, “Symbolically Analyzing Security Protocols using TAMARIN,” ACM SIGLOG News, vol. 4, no. 3, pp. 32-41, Oct. 2017. [Online]. Available: <https://ethz.ch/content/dam/ethz/special-interest/infk/inst-infsec/information-security-group-dam/research/publications/pub2017/siglog-tamarin.pdf>.
- [3] D. Basin, C. Cremers, J. Dreier, R. Sasse. Tamarin: Verification of Large-Scale, Real World, Cryptographic Protocols. IEEE Security and Privacy Magazine, In press, (10.1109/msec.2022.3154689). (hal-03586826)
- [4] National Security Agency (NSA), “Cryptographic System,” Declassified Friedman Documents. [Online]. Available: <https://www.nsa.gov/portals/75/documents/news-features/declassified-documents/friedman-documents/patent-equipment/folder-545/41781159082021.pdf>.
- [5] F. Rubin, “One-time pad cryptography,” Cryptologia, vol. 20, no. 4, pp. 359–364, 1996, doi: 10.1080/0161-119691885040.
- [6] Cervasato, “The Dolev-Yao intruder is the most powerful attacker,” ITT Industries, Inc., Alexandria, VA, USA. [Online]. Available: <https://www.csl.sri.com/users/filiano/>

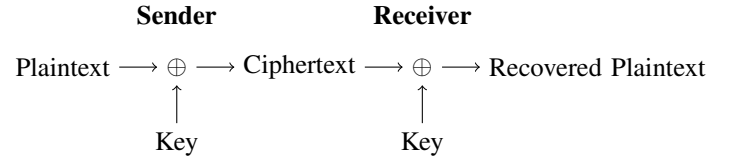


Fig. 1. One-Time Pad (OTP) encryption and decryption flow. Sender XORs the message and key to create ciphertext. Receiver XORs ciphertext and key to recover the message.

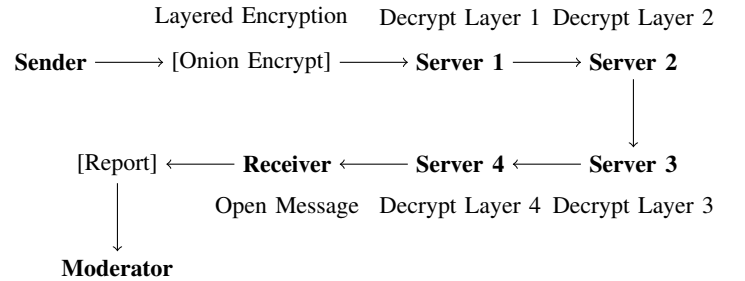


Fig. 2. Onion Franking flow modeled in Tamarin: sender onion-encrypts the message, servers sequentially peel layers, receiver recovers the message and generates an abuse report, moderator verifies the report.

IX. INDEX

Proof of Incorrect Indexing in Tamarin Paper for Construction V.1: We consider c_3 , as created by the loop in the Send function [1]. As written in the paper, $c_{3,i} = Enc(pk_i, (c_{3,i-1}, r_i))$ and makes up st_{i-1} . We know this because the Process function of each server takes st_{i-1} as an input.

Then, $Dec(ski_i, c_{3,i}) = (c_{3,i-1}, r_i)$ in S'_i ’s Process function ($c_{3,i-1} = c'_3$, to match notation exactly). Then, we get $st_i = (c'_3, mrt') = (c_{3,i-1}, mrt')$ (mrt may be ignored) and return st_i on the last two lines of Server i ’s Process function.

Now, let's consider $c_{3,i-1}$. By definition, $c_{3,i-1} = \text{Enc}(pk_{i-1}, (c_{3,i-2}, r_{i-1}))$. By the same reasoning applied before, $c_{3,i-1}$ makes up st_{i-2} .

Then, we have that $c_{3,i-1}$ makes up both st_{i-2} and st_i , which is a contradiction.

A more intuitive way to see this indexing error is that since we're sending something through the servers in the order $S_1 \rightarrow S_2 \rightarrow \dots \rightarrow S_N$, we should encrypt things in the manor $c_3 = \text{Enc}_1(\text{Enc}_2(\dots(\text{Enc}_N(z))))$, where $\text{Enc}_i(x) = \text{Enc}(pk_i, x)$ and z is the information we mean to send through the servers. All this proof means to claim is that the servers and the encryption have been indexed in contradictory ways.

```

Send(pk,  $k_r$ ,  $m$ )
    ⋮
     $c_{3,N} \leftarrow \epsilon$ 
    for  $i \in \{N, \dots, 1\}$  :
         $c_{3,i} \leftarrow \text{Enc}(\mathbf{pk}_i, (c_{3,i+1}, r_i))$ 
     $c_3 \leftarrow c_{3,1}$ 
    return  $c_1, c_2, c_3$ 

```

$$S_1 \rightarrow S_2 \rightarrow S_2 \rightarrow S_4$$

$$\text{Enc}_i(x) = \text{Enc}(pk_i, x)$$

$$c_3 = \text{Enc}_4(\text{Enc}_3(\text{Enc}_2(\text{Enc}_1(z))))$$

$$c_3 = \text{Enc}_1(\text{Enc}_2(\text{Enc}_3(\text{Enc}_4(z))))$$